

Claude & GitHub Agents

The Definitive Best Practices Guide

Skills · Hooks · Plugins · MCP · Token & Cost Optimization

April 2026 | Based on Anthropic Official Docs & Community Research

60–80%

Cost Reduction
with optimization

12

Hook Lifecycle
Events

190+

Agentic Coding
Patterns Available

This guide consolidates official Anthropic documentation, community research, and production-proven patterns to help developers and organizations build efficient, scalable, and cost-effective AI agent systems with Claude.

Table of Contents

01	Architecture & Concepts	Agent Skills, MCP, Subagents, Hooks — how they fit together
02	Agent Skills — Build & Best Practices	SKILL.md anatomy, dynamic loading, compaction budgets
03	Hooks — Lifecycle Automation	12 events, handler types, security gates, async patterns
04	Plugins & MCP Integration	MCP servers, tool design, OAuth, security model
05	CLAUDE.md & Context Engineering	System files, project rules, .claudeignore
06	Token & Cost Optimization	Caching, compaction, model tiering, 60-80% savings
07	GitHub Actions Integration	CI/CD patterns, automated code review, PR workflows
08	Security Best Practices	CVEs, credential protection, trust model, ZDR
09	Subagents & Multi-Agent Patterns	Orchestration, parallel patterns, agent teams
10	Reference Repositories & Resources	awesome-claude-code, skills repo, cookbooks

Architecture & Concepts

Claude's agent ecosystem is composed of four complementary primitives: **Skills**, **Hooks**, **MCP Servers**, and **Subagents**. Understanding how these interact is the foundation for building efficient, scalable systems.

The Four Primitives

Primitive	What It Is	When To Use	Token Impact
Skills	Folder of SKILL.md + scripts loaded on demand	Domain knowledge, document tasks, org workflows	Load once, shared budget (~25K tokens)
Hooks	Shell/HTTP commands at lifecycle events	Deterministic enforcement, security gates, formatting	Zero — runs outside model context
MCP Servers	External tool providers via Model Context Protocol	API integrations, databases, cloud services	Tool definitions added to context
Subagents	Forked Claude instances with scoped tools	Parallel research, isolating heavy file reads	Separate context — returns summary only

How They Work Together

The recommended mental model (from production experience): **CLAUDE.md** for things Claude gets wrong **without it** (keep under 80 lines). **Skills** for domain knowledge, loaded on demand. **Hooks** for things that must **always happen** — deterministically. **Subagents** for exploration — keep the main session clean by delegating heavy file reads.

■ Core Design Principle

Every tool, skill, and hook must justify its existence. Self-contained, non-overlapping, and purpose-specific design is essential. Overlapping skills cause context bloat.

The Agent Skills Open Standard

Anthropic published Agent Skills as an **open standard** (December 2025) for cross-platform portability. The same SKILL.md format works across Claude.ai, Claude Code, the Anthropic API, and third-party tools like Agno and Cursor. A Skill directory contains:

- **SKILL.md** — The main instructions file with frontmatter metadata
- **Scripts** — Python, bash, or Node scripts for deterministic operations
- **Resources** — Reference files, templates, examples
- **Images** — Diagrams or visual references Claude can see

Agent Skills — Build & Best Practices

Agent Skills are the primary way to give Claude domain expertise. They package instructions, scripts, and resources into composable, reusable units — like an onboarding guide for a new hire.

SKILL.md Anatomy

Every skill starts with a SKILL.md file. Here is the full structure:

```
---
name: financial-analysis
description: Analyze financial statements and generate Excel reports.
            Use PROACTIVELY when user mentions revenue, P&L, or balance sheets.
invocation: auto          # auto | explicit | none
agent: Explore            # Explore | Plan | general-purpose | custom-name
context: fork             # fork | inline
allowed-tools: Bash(python3 *), Read
---

# Financial Analysis Skill

## Overview
You are an expert financial analyst. When invoked, follow these steps exactly:

1. Load the provided file using the Read tool
2. Extract: Revenue, COGS, Gross Margin, EBITDA
3. Generate an Excel report using the xlsx skill
4. Save to /tmp/analysis output.xlsx

## Important Rules
- NEVER invent numbers – only use data present in the source file
- Always include a confidence score (0-100) in the output
- Flag anomalies with [ANOMALY] prefix

## Dynamic Context
Current date: !`date +%Y-%m-%d`
Project root: !`pwd`
```

Skill Description Engineering

The **description** field is the most critical part of any skill. Claude uses it to decide whether to auto-load the skill. A poor description means the skill never fires. An over-broad description causes context bloat on every request.

Pattern	Bad Example	Good Example
Trigger clarity	'Handles documents'	'Use PROACTIVELY when user asks to create Excel (.xlsx) files or analyze spreadsheet data'
Negative examples	(none)	'Do NOT use for CSV-only tasks or simple text tables'

Scope specificity	'Financial stuff'	'Covers income statements, balance sheets, and cash flow — NOT general math'
Action verbs	'For reports'	'Generates, analyzes, extracts, transforms'

Invocation Modes

- **auto** — Claude loads the skill automatically when the description matches. Best for broad capabilities.
- **explicit** — User must type the slash command (e.g., /financial-analysis). Best for heavyweight or sensitive operations.
- **none** — Skill is a library only, invoked by other skills via dynamic context injection.

Context Compaction Budget

When Claude Code auto-compacts the conversation, it re-attaches the most recently invoked skills. The shared budget is **25,000 tokens**, filled starting from the most recent skill. Older skills may be dropped entirely after compaction.

■■ Compaction Warning

Claude does NOT re-read the skill file on later turns. Write guidance as standing instructions, not one-time steps. If a skill stops influencing behavior after the first response, strengthen its description or use hooks for deterministic enforcement.

Pre-built Anthropic Skills

Skill ID	Purpose	API Beta Header
pptx	Create professional PowerPoint presentations	skills-2025-10-02
xlsx	Generate Excel spreadsheets with formulas/charts	skills-2025-10-02
docx	Create and edit Word documents with rich formatting	skills-2025-10-02
pdf	Analyze and extract information from PDFs	skills-2025-10-02
claude-api	Up-to-date API reference, SDK docs, best practices	Bundled in Claude Code

API Usage — Container Reuse Pattern

Reuse containers across multi-turn conversations to avoid re-loading skill context:

```
# Python – reuse container across turns
import anthropic
client = anthropic.Anthropic()

# First request – container is created
```

```

response = client.beta.messages.create(
    model='claude-sonnet-4-6',
    max_tokens=4096,
    betas=['code-execution-2025-08-25', 'skills-2025-10-02'],
    container={'skills': [{'type': 'anthropic', 'skill id': 'xlsx', 'version': 'latest'}]},
    messages=[{'role': 'user', 'content': 'Analyze sales data'}],
    tools=[{'type': 'code execution 20250825', 'name': 'code execution'}]
)
container_id = response.container.id # Save this!
# Subsequent requests – reuse container (no skill reload cost)
response2 = client.beta.messages.create(
    model='claude-sonnet-4-6',
    max_tokens=4096,
    betas=['code-execution-2025-08-25', 'skills-2025-10-02'],
    container={'id': container_id}, # Reuse!
    messages=[..., {'role': 'user', 'content': 'What was total revenue?'}]
)

```

Hooks — Lifecycle Automation

Hooks are user-defined commands that execute automatically at specific points in Claude Code's lifecycle. They transform best-practice documentation into **enforced, deterministic rules** — the only way to guarantee behavior regardless of model choice.

The 12 Lifecycle Events

Event	When it Fires	Can Block?	Primary Use Cases
UserPromptSubmit	After user submits, before Claude processes	Yes	Inject skill hints, prompt augmentation
PreToolUse	Before any tool executes	Yes ■	Security gates, file protection, validation
PostToolUse	After tool completes	No	Auto-format, lint, compliance logging
Stop	When Claude finishes its turn	No	Notifications, test runners, summaries
SubagentStop	When a subagent completes	No	Aggregate results, subagent monitoring
PreCompact	Before context is compacted	No	Conversation backup, state snapshot
PostCompact	After compaction completes	No	Reload critical context
Notification	When Claude needs human input	No	Slack/email alerts, push notifications
AgentStart	When an agent session begins	No	Environment setup, logging init
AgentStop	When agent session ends	No	Cleanup, final reports, cost summary
Error	On tool execution errors	No	Error logging, fallback actions
StatusLine	Continuous context metrics	N/A	Token burn rate, backup triggers

Handler Types

- **Command hooks** — Shell commands. Receive JSON via stdin, return via exit code. Best for local scripts.
- **HTTP hooks** — POST to a web server. Enables team-wide policy servers. Non-2xx = non-blocking (use decision:'block' in body).
- **Prompt hooks** — Send to a Claude model for semantic evaluation. Use \$ARGUMENTS placeholder.
- **Agent hooks** — Spawn subagents with Read/Grep/Glob access. Best for deep verification.

■ **Async hooks** — Add `async:true` to run in background without blocking Claude (released Jan 2026).

PreToolUse — The Security Gate

PreToolUse is the most powerful hook — the only one that can block tool execution. It is the foundation of all security enforcement in Claude Code.

```
# .claude/settings.json — multi-pattern security gate
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [{
          "type": "command",
          "command": "python3 ~/.claude/hooks/security_gate.py"
        }]
      },
      {
        "matcher": "Edit",
        "hooks": [{
          "type": "command",
          "command": "bash ~/.claude/hooks/protect_sensitive.sh"
        }]
      }
    ],
    "PostToolUse": [{
      "matcher": "Edit",
      "hooks": [{"type": "command", "command": "pre-commit run --files $CLAUDE_FILE_PATHS"}]
    }],
    "Stop": [{"hooks": [{"type": "command", "command": "python3 notify_slack.py", "async": true}]}]
  }
}
```

Token Preprocessing Hook Pattern

Instead of Claude reading a 10,000-line log file to find errors (tens of thousands of tokens), a PreToolUse hook can grep for ERROR and return only matching lines, **reducing context from tens of thousands of tokens to hundreds**.

```
#!/usr/bin/env python3
# PreToolUse hook: filter test output to show only failures
import json, sys, subprocess

hook_input = json.load(sys.stdin)
tool_name = hook_input.get('tool_name', '')
```



```

tool input = hook input.get('tool input', {})
if tool name == 'Bash' and 'pytest' in tool input.get('command', ''):
    # Run the command but filter output
    result = subprocess.run(tool input['command'], shell=True, capture output=True, text=True)
    # Return only FAILED lines – massive token reduction
    failures = [l for l in result.stdout.split('\n') if 'FAILED' in l or 'ERROR' in l]
    print(ison.dumps({'decision': 'approve', 'output override': '\n'.ioin(failures)}))
else:
    print(json.dumps({'decision': 'approve'}))

```

Context-Aware Backup with StatusLine

The StatusLine hook is the only mechanism receiving live context metrics. Use it to trigger backups at token thresholds before quality degrades from context rot:

```

// statusline-monitor.mjs
const AUTOCOMPACT BUFFER = 33000; // Fixed buffer tokens
const BACKUP_THRESHOLD = 0.30;    // Backup when 30% free
const metrics = JSON.parse(process.env.CLAUDE STATUS METRICS || '{}');
const { tokens used, tokens max } = metrics;
const freeRatio = 1 - (tokens used / tokens max);
if (freeRatio < BACKUP_THRESHOLD) {
    // Trigger asvnc backup
    import('./backup-core.mjs').then(m => m.saveBackup(metrics));
}
// Display: [!] 28% free (720K/1M) -> .claude/backups/3-backup.md
process.stdout.write(`[!] ${((freeRatio*100).toFixed(0))}% free`);

```

Plugins & MCP Integration

The Model Context Protocol (MCP) is an open standard for connecting Claude to external tools, databases, and services. Claude Code supports MCP natively; the Anthropic API supports MCP servers via the `mcp_servers` parameter in API calls.

MCP Architecture

- **MCP Server** — A process exposing tools, resources, and prompts via a standardized API
- **Tool** — A callable function with explicit parameters and a concise description
- **Resource** — Read-only content (files, DB records) Claude can reference
- **Prompt** — Reusable prompt templates the MCP server provides

MCP Tool Design Best Practices

Tool descriptions are the single most important factor in MCP performance:

Self-contained

Each tool must have a complete, unambiguous description. Do not rely on Claude's world knowledge to fill gaps.

Non-overlapping

Never create two tools that could handle the same input. Claude will pick one arbitrarily, causing inconsistency.

Explicit parameters

All parameters must have clear types, constraints, and examples. Avoid optional parameters when possible.

Negative examples

Include what NOT to use the tool for. These are critical — they define the tool's boundaries.

PROACTIVELY signals

Prefix the description with 'Use PROACTIVELY when...' to influence auto-selection.

MCP in the API (Claude-in-Claude Pattern)

```
// AI-powered artifact calling Claude API with MCP servers
const response = await fetch('https://api.anthropic.com/v1/messages', {
  method: 'POST',
  headers: {'Content-Type': 'application/json'},
  body: JSON.stringify({
    model: 'claude-sonnet-4-6',
    max tokens: 4096,
    mcp_servers: [
      { type: 'url', url: 'https://gmail.mcp.claude.com/mcp', name: 'gmail' },
      { type: 'url', url: 'https://acal.mcp.claude.com/mcp', name: 'acal' }
    ],
  })
})
```

```
    messages: [{ role: 'user', content: userMessage }]
  })
});
// Process response – handle mcp tool use blocks by TYPE not position
const toolResults = data.content
  .filter(item => item.type === 'mcp tool result')
  .map(item => item.content?.[0]?.text || '');
```

Claude Code Plugin Installation

Claude Code plugins are installed via the official Anthropic skills marketplace or manually. The same skill format works as a plugin:

```
# Install from marketplace
claude plugins install @anthropic/skills-xlsx

# Manual install to project
mkdir -p .claude/skills/my-skill

# Add SKILL.md to .claude/skills/my-skill/SKILL.md

# Manual install globally (available in all projects)
mkdir -p ~/.claude/skills/my-skill

# Verify skills are detected
claude skills list
```

■ MCP Security Model

MCP servers run as external processes. Apply least-privilege: restrict each server's allowed tools to the minimum needed. Use the `allowedEnvVars` field in HTTP hooks to prevent environment variable leakage. Never expose production credentials in MCP configs.

CLAUDE.md & Context Engineering

Context Engineering is the discipline of optimizing token utility within the constraints of LLM context windows. The CLAUDE.md file and .claudeignore are the primary tools for shaping what Claude knows about your project.

CLAUDE.md Best Practices

Keep it under 80 lines

CLAUDE.md is loaded on every turn. Every line costs tokens on every message. Be ruthless about what goes here.

Only corrections, not tutorials

Put only things Claude gets wrong without it. If Claude already knows it, don't repeat it.

Architecture, not implementation

Describe your project's key directories, naming conventions, and non-obvious decisions.

Reference skills, not inline

Instead of embedding domain knowledge in CLAUDE.md, create a skill and reference it: 'See /financial-analysis for P&L; rules.'

Absolute commands

Use imperative language: 'ALWAYS run npm test before committing.' Hedged language ('prefer', 'try to') reduces compliance.

CLAUDE.md Template

```
# Project: MvApp
## Architecture
- Frontend: Next.js 14 + TypeScript (src/app/)
- Backend: Node.js + Express (src/api/)
- DB: PostgreSQL via Prisma (prisma/schema.prisma)
- Auth: NextAuth.js – DO NOT modify auth logic without /auth-review skill
## Key Commands
- `npm run dev` – Start dev servers (port 3000)
- `npm run test` – Jest + RTL (ALWAYS run before committing)
- `npm run type-check` – TypeScript strict mode
## Non-Obvious Rules
- All API responses MUST use the ApiResponse<T> wrapper (src/types/api.ts)
- Never import directly from 'lodash' – use src/utils/lodash.ts re-exports
- Environment secrets live in .env.local – NEVER commit this file
## Code Standards
- Components under 200 lines; split if larger
- JSDoc for all public functions
```

- Test coverage minimum 80%

.claudeignore — Context Scoping

The .claudeignore file works like .gitignore but for Claude's file access. Excluding irrelevant directories is one of the highest-leverage token optimizations:

```
# .claudeignore
node modules/
dist/
build/
.next/
*.log
*.lock
coverage/
.git/
# Exclude large data files Claude doesn't need to read
data/raw/
*.csv
*.parquet
# Exclude generated files
src/generated/
*.d.ts
```

■ Context Rot Warning

Research shows all models experience performance degradation with long contexts. Quality degrades at 20-40% full — NOT 80-90% as you might expect. Set `CLAUDE_AUTOCOMPACT_PCT_OVERRIDE=60` so compaction kicks in before degradation starts.

Token & Cost Optimization

The combination of structural optimizations, prompt caching, model tiering, and context management can reduce API costs by **60-80%** for production agent applications. This chapter covers every lever available.

60-80%

Cost reduction
possible

\$13

Avg daily cost
per developer

40-70%

Reduction from
basic strategies

Tier 1: Prompt Caching (Biggest Win)

Prompt caching is the single largest cost reduction lever. Repeated content (system prompts, skill context, large documents) is cached automatically. With cache-aware rate limits (2026), cached tokens don't count against ITPM limits.

```
# Enable prompt caching on system prompt
response = client.messages.create(
    model='claude-sonnet-4-6',
    max_tokens=4096,
    extra_headers={'anthropic-beta': 'token-efficient-tools-2025-02-19'},
    system=[{
        'type': 'text',
        'text': 'You are a customer support agent. ' + LARGE KNOWLEDGE BASE,
        'cache control': {'type': 'ephemeral'} # Cache this!
    }],
    messages=conversation history
)

# cache read input tokens shown in usage – these are cheap
print(response.usage) # cache_read: 45000, cache_write: 200, input: 200
```

Tier 2: Token-Efficient Tool Use

Enable with one beta header. Reduces tool definition overhead significantly. Available for Claude Sonnet 4.6, Opus 4.6, and Haiku 4.5:

```
extra_headers={'anthropic-beta': 'token-efficient-tools-2025-02-19'}
# Combined with caching: 60-80% total cost reduction
```

Tier 3: Model Tiering Strategy

The most common mistake is using Opus for every task. Route by complexity:

Task Type	Recommended Model	Why
High-level planning, architecture, final review	Claude Opus 4.6	Complex reasoning, rare execution
Feature implementation, code generation, debugging	Claude Sonnet 4.6	Best code quality / cost ratio
Linting, formatting, simple transforms, status checks	Claude Haiku 4.5	Fast, cheap, sufficient for simple tasks
Hook evaluation, prompt classification	Claude Haiku 4.5	Near-instant, sub-cent per call

Tier 4: Context Management

- **/clear between tasks** — Stale context wastes tokens on every subsequent message. Use /rename first so you can /resume.
- **/compact with instructions** — Run '/compact Focus on code samples and API usage' to preserve relevant content.
- **CLAUDE_AUTOCOMPACT_PCT_OVERRIDE=60** — Trigger compaction at 60% full, before quality degrades.
- **.claudeignore** — Exclude node_modules, dist, logs. Prevents accidental large file reads.
- **Subagents for exploration** — File reads in subagents return summaries to main context, not raw content.
- **Specific file references** — 'Read src/auth/login.ts' beats 'Read the auth module' (avoids directory scan).

Tier 5: Preprocessing Hooks

Hooks run outside model context — zero token cost. They are the most efficient way to pre-process data before it enters Claude's context window:

- Filter 10,000-line logs to 50 error lines before Claude sees them
- Pre-summarize large JSON API responses to key fields only
- Convert binary files to text summaries
- Inject only relevant skill context based on prompt keywords (SkillActivationHook pattern)

Tier 6: Automation Safeguards

```
# Prevent runaway costs in CI/CD automation
claude --max-turns 15 --timeout-minutes 30 'Run full test suite'

# Set workspace spend limits in Claude Console
# Admins: Console > Workspaces > Claude Code > Spend Limit
# Track: /cost (API users) or /stats (subscribers)
```

■ Cost Tracking Stack

Use ccflare or ccxray for real-time dashboards. claude-code-usage-monitor for terminal burn-rate tracking. Set ANTHROPIC_BUDGET_TOKEN env var for automated budget enforcement.

GitHub Actions Integration

Claude Code integrates natively with GitHub Actions via the official anthropics/claude-code-action. This enables automated code review, PR summarization, issue triage, and full agentic CI/CD workflows.

Basic CI/CD Integration

```
# .github/workflows/claude-review.yml
name: Claude Code Review
on:
  pull request:
    types: [opened, synchronize]
jobs:
  review:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pull-requests: write
    steps:
      - uses: actions/checkout@v4
      - uses: anthropics/claude-code-action@v1
        with:
          anthropic_api_key: ${ secrets.ANTHROPIC_API_KEY }
          model: claude-sonnet-4-6
          max_turns: 10
          timeout_minutes: 15
          prompt: |
            Review this PR for:
            1. Security vulnerabilities
            2. Breaking API changes
            3. Missing test coverage
            4. Performance regressions
            Post a detailed review comment.
```

GitHub Actions Best Practices

- **Pin model versions** — Use claude-sonnet-4-6 not latest to prevent unexpected behavior changes.
- **Set max_turns and timeout_minutes** — Always cap automation to prevent runaway costs.
- **Use Haiku for triage** — Route issue labeling and simple checks to Haiku; use Sonnet only for complex review.
- **Cache action dependencies** — Use actions/cache for node_modules to avoid reinstall on every run.
- **Scope permissions minimally** — Only grant pull-requests: write if the action posts comments.
- **Store API keys as secrets** — Never hardcode ANTHROPIC_API_KEY in workflow files.

- **Use concurrency groups** — Prevent multiple reviews running simultaneously on the same PR.

Agentic Workflow Patterns

Combine Claude Code with GitHub Actions for fully autonomous workflows:

Pattern	Description	Recommended Model
PR Summarizer	Auto-generate PR descriptions from diff + commits	Haiku 4.5
Security Scanner	Run SAST + Claude analysis on every push	Sonnet 4.6
Auto-Triage	Label and assign issues based on content	Haiku 4.5
Dependency Reviewer	Analyze new dependencies for risk	Sonnet 4.6
Release Notes	Generate CHANGELOG from merged PRs	Sonnet 4.6
Architecture Review	Deep review of large PRs before merge to main	Opus 4.6

■ **Cost Tip for CI/CD**

Use SKIP_REVIEW label to skip Claude review on documentation-only PRs. Add a check in your workflow: `if contains(github.event.pull_request.labels.*.name, 'skip-review') → skip`. This alone can reduce CI costs by 20-30% for active repos.

Security Best Practices

Claude Code runs agentic loops with real system access. Security must be designed-in, not bolted-on. This chapter covers known CVEs, structural risks, and enforcement patterns.

Known CVEs — Patch Status

CVE	CVSS	Description	Fix
CVE-2025-59536	8.7	Hook/MCP init runs before trust dialog — code executes before user consent	Fixed v1.0.111
CVE-2025-55284	7.1	API key exfiltration via DNS (prompt injection abuses dig/nslookup)	Fixed v1.0.4
CVE-2025-52882	8.8	VS Code WebSocket auth bypass — malicious sites can connect, potential RCE	Fixed VS Code ext v1.0.24+
CVE-2026-21852	5.3	API key exfiltration via malicious ANTHROPIC_BASE_URL config	Fixed v1.1.x
CVE-2026-33068	7.7	Workspace trust dialog bypass via repo-controlled settings files	Fixed v2.1.53

■ Critical Rule

NEVER use `--dangerously-skip-permissions` in directories you do not fully control. This flag disables all safety checks and is the most common cause of security incidents in CI/CD pipelines with untrusted repository content.

Credential Protection via Hooks

Use PreToolUse hooks to block credential file access deterministically:

```
#!/usr/bin/env bash
# protect_credentials.sh — blocks access to secrets files
INPUT=$(cat)
FILE_PATH=$(echo $INPUT | python3 -c 'import sys,json; d=json.load(sys.stdin); print(d.get("tool input",{}).get("path",""))')
PROTECTED_PATTERNS=(' .env' '.env.local' '*.pem' '*.key' 'id_rsa' '.ssh/')
for pattern in "${PROTECTED_PATTERNS[@]%%}"; do
  if [[ "$FILE_PATH" == *"$pattern"* ]]; then
    echo '{"decision": "block", "reason": "Access to credential file blocked by policy"}'
    exit 0
  fi
done
echo '{"decision": "approve"}'
```

Skills Security Model

- **Only use skills from trusted sources** — Skills execute code in your environment. A malicious skill can exfiltrate data.
- **Audit all files in a skill** — Review SKILL.md, scripts, and any resources. Look for unexpected network calls.
- **Skills are not ZDR-eligible** — Skill definitions and execution data are retained per Anthropic's standard policy.
- **Pin skill versions** — Don't use 'latest' in production. Pin to a specific version you've audited.
- **Isolate sensitive skills** — Skills that access production data should only be enabled in controlled environments.

AI-Generated Code Risk

Community studies report AI-generated code has 1.5-2x higher security vulnerability rates than human-written code. Apply extra scrutiny to auth flows, payment logic, and data mutation operations generated by Claude.

Subagents & Multi-Agent Patterns

Subagents are forked Claude instances that run in parallel or sequence, each with scoped tools and context. They are the key to keeping the main session clean and costs low.

Core Subagent Patterns

Subagent Orchestration

A master agent breaks a task into subtasks and delegates each to specialized subagents. Results are summarized and returned to main context.

Progressive Skills

Agents load skills incrementally as the task evolves — avoiding upfront context bloat from loading all possible skills.

Parallel Tool Calling

Multiple independent operations (read 5 files, call 3 APIs) run concurrently inside a single turn.

Master-Clone Architecture

A master agent with full context spawns lightweight clones for execution. Clones have minimal context and cheap model selection.

Wizard Workflows

Multi-step guided workflows where each step is a specialized subagent. The wizard hands off state between steps.

Explore Agent

Read-only subagent optimized for codebase exploration. Returns summaries, not raw file contents, protecting main context.

Subagent Skill Pattern

```
---
name: deep-research
description: Research a topic thoroughly in the codebase without polluting main context.
            Use when understanding architecture, tracing data flow, or mapping dependencies.
context: fork
agent: Explore
---

Research $ARGUMENTS thoroughly:
1. Find relevant files using Glob and Grep
2. Read and analyze the code
3. Map all dependencies and callers
4. Summarize findings with specific file:line references

Return ONLY a structured summary – do not include raw file contents.
```

Multi-Agent Cost Optimization

- **Delegate exploration to subagents** — File reads in main context are expensive. Subagents return summaries.
- **Use Haiku for leaf nodes** — Subagents doing simple transforms should use Haiku, not Sonnet.
- **Scope MCP tools per subagent** — Only give each subagent the tools it needs. Fewer tools = smaller context.
- **Return summaries, not data** — Subagents should summarize findings. Raw data return defeats the purpose.
- **Parallelize where independent** — Five sequential file reads vs. five parallel reads: same tokens, 5x faster.

Agent Teams Pattern (Claude Code)

Claude Code supports structured agent teams through `.claude/agents/` directory. Each agent file defines a specialized role, tools, and capabilities:

```
# .claude/agents/security-reviewer.md
---
name: security-reviewer
description: Specialized agent for security code review.
             Invoked automatically on any changes to auth/, payments/, or api/ directories.
model: claude-opus-4-6 # Use best model for security review
allowed-tools: Read, Grep, Glob
---
You are a security expert. Review code for:
- SQL injection, XSS, CSRF vulnerabilities
- Insecure deserialization
- Hardcoded credentials
- Improper authentication/authorization
ALWAYS flag findings with severity: [CRITICAL|HIGH|MEDIUM|LOW]
```

Reference Repositories & Resources

The Claude Code ecosystem has grown rapidly. Here are the highest-quality resources verified as of April 2026, organized by category.

Essential GitHub Repositories

Repository	Description	Stars	Key Contents
hesreallyhim/awesome-claude-code	Curated list of skills, hooks, slash-commands, orchestrators, plugins	30.9K ■	Everything — start here
anthropics/skills	Official Anthropic skills marketplace and open standard	Official	pptx, xlsx, docx, pdf, claude-api skills
anthropics/claude-code-action	Official GitHub Actions integration	Official	CI/CD workflows, examples
anthropics/anthropic-quickstarts	Demo projects with standardized workflows	Official	3 complete agent demos
ThibautMelen/agentic-patterns	190+ agentic patterns with Mermaid diagrams	Community	Orchestration, parallel, master-clone
disler/context-priming	Systematic context priming for different project types	Community	CLAUDE.md templates, priming commands

Cost Tracking & Monitoring Tools

Tool	Type	Key Feature
ccflare / better-ccflare	Web dashboard	Beautiful UI, comprehensive metrics, Docker support
ccxray	Proxy + dashboard	Transparent proxy, Miller-column interface, session grouping
claude-code-usage-monitor	Terminal	Real-time burn rate, predictions, subscription-aware
claudia-statusline	Terminal statusline	Rust-based, SQLite persistence, XDG-compliant, themes
CC Usage (ryoppippi)	CLI analyzer	Analyzes local logs, nice dashboard, cost breakdown

Official Documentation Links

- **Agent Skills Overview** — platform.claude.com/docs/en/agents-and-tools/agent-skills/overview
- **Claude Code Hooks** — code.claude.com/docs/en/hooks
- **Claude Code Skills** — code.claude.com/docs/en/skills

- **Cost Management** — code.claude.com/docs/en/costs
- **API Skills Guide** — platform.claude.com/docs/en/build-with-claude/skills-guide
- **Anthropic Quickstarts** — github.com/anthropics/anthropic-quickstarts
- **Skills Cookbook** — github.com/anthropics/anthropic-cookbook/tree/main/skills
- **Prompt Engineering** — docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview
- **MCP Documentation** — spec.modelcontextprotocol.io
- **Agentic Patterns Academy** — anthropic.skilljar.com/introduction-to-agent-skills

Quick-Start Checklist

Follow this order when setting up a new Claude Code project:

1. Create CLAUDE.md (under 80 lines, corrections only, architecture overview)
2. Add .claudeignore (exclude node_modules, dist, logs, generated files)
3. Set CLAUDE_AUTOCOMPACT_PCT_OVERRIDE=60 in your environment
4. Install pre-built skills as needed: pptx, xlsx, docx, pdf
5. Add a PreToolUse security hook blocking credential file access
6. Add a PostToolUse formatting hook (prettier/black on edit)
7. Create domain-specific skills for your team's workflows
8. Set up ccflare or ccxray for cost visibility
9. Configure workspace spend limits in Claude Console
10. Add token-efficient-tools beta header to all API calls
11. Enable prompt caching on system prompts and large context
12. Pin model versions in production (never use 'latest')

■ Summary

The biggest wins are structural, not model selection. Get your context architecture right first (.claudeignore, .claudeignore, skills, hooks), then optimize caching and tool efficiency. Developers consistently achieve 60-80% cost reduction with these techniques — often in a single day of setup.